

GEESOLVER: A Generic, Efficient, and Effortless Solver with Self-Supervised Learning for Breaking Text Captchas

Ruijie Zhao^{†§}, Xianwen Deng^{†§*}, Yanhao Wang[‡], Zhicong Yan[†], Zhengguang Han^{††},
Libo Chen[†], Zhi Xue^{†*}, and Yijun Wang^{†*}

[†]Shanghai Jiao Tong University [‡]QI-ANXIN ^{††}QI-AN Pangu (Shanghai) InfoTech Co., Ltd.
[†]{ruijiezhao, 2594306528, zhicongy, bob777, zxue, ericwyj@sjtu.edu.cn},
[‡]{wangyanhao136@gmail.com}, ^{††}{hanzhengguang@qianxin.com}

Abstract—Although text-based captcha, which is used to differentiate between human users and bots, has faced many attack methods, it remains a widely used security mechanism and is employed by some websites. Some deep learning-based text captcha solvers have shown excellent results, but the labor-intensive and time-consuming labeling process severely limits their viability. Previous works attempted to create easy-to-use solvers using a limited collection of labeled data. However, they are hampered by inefficient preprocessing procedures and inability to recognize the captchas with complicated security features.

In this paper, we propose GEESOLVER, a generic, efficient, and effortless solver for breaking text-based captchas based on self-supervised learning. Our insight is that numerous difficult-to-attack captcha schemes that “damage” the standard font of characters are similar to image masks. And we could leverage masked autoencoders (MAE) to improve the captcha solver to learn the latent representation from the “unmasked” part of the captcha images. Specifically, our model consists of a ViT encoder as latent representation extractor and a well-designed decoder for captcha recognition. We apply MAE paradigm to train our encoder, which enables the encoder to extract latent representation from local information (i.e., without masking part) that can infer the corresponding character. Further, we freeze the parameters of the encoder and leverage a few labeled captchas and many unlabeled captchas to train our captcha decoder with semi-supervised learning.

Our experiments with real-world captcha schemes demonstrate that GEESOLVER outperforms the state-of-the-art methods by a large margin using a few labeled captchas. We also show that GEESOLVER is highly efficient as it can solve a captcha within 25 ms using a desktop CPU and 9 ms using a desktop GPU. Besides, thanks to latent representation extraction, we successfully break the hard-to-attack captcha schemes, proving the generality of our solver. We hope that our work will help security experts to revisit the design and availability of text-based captchas. The code is available at <https://github.com/NSSL-SJTU/GeeSolver>.

[§]Equal contribution. ^{*}Corresponding authors.

1. Introduction

Captcha [1], which stands for “Completely Automated Public Turing Test to Tell Computers and Human Apart”, has evolved as a powerful technology for distinguishing computer programs from humans. Although numerous alternatives to text-based captcha have been proposed [2]–[7], text-based captcha remains the extensively used security solution by many websites (e.g., Google, Yandex, and Microsoft) and the web services of IoT devices (e.g. ASUS router). Thus, existing text-based captcha schemes should be thoroughly evaluated for their ability to withstand various attacks.

Many strategies for bypassing text-based captchas have been proposed during the previous decade. One early approach proposed is to break text-based captchas using segmentation-based approaches [8]–[16]. However, when segmentation-resistant measures (e.g., character sticking and overlapping) are introduced to text-based captchas, segmentation-based assaults become obsolete. Besides, some DL-based approaches [17]–[22] have shown outstanding results, but they require a large number of labeled samples for training. The labor-intensive and time-consuming labeling process severely limits the viability of such approaches. Ye *et al.* [23] and Tian *et al.* [24] noticed this challenge and sought to develop effortless solvers that only require limited labeled data. However, their solutions are not efficient and generic enough as they design complex and time-consuming preprocessing procedures to eliminate noisy backgrounds, and fail to recognize the captchas with the latest complicated features, such as *Google* captchas, which introduce the security feature of crowding characters together (CCT). As can be observed, in recent years, the deployment of more complicated security features is on the rise, a solution that is not generic can be easily outdated and defended. Given the difficulties stated above, proposing an effectively DL-based solver to break various captchas is still necessary. The solver should conform to three constraints: general solution, efficient attack, and effortless labeling.

However, depending on previous works, it is hard to satisfy all these three requirements simultaneously, making

us look for another solution. We observe that numerous difficult-to-attack captcha schemes greatly “damage” the standard font of characters via sophisticated security features (e.g., crowding, overlapping, rotation, distortion, occlusion). Even for humans, the relevant characteristics can only be inferred from local representative information. As a result, this local representative information should be regarded as the key to accurately identifying the captcha. It also gives us an intuitive solution for bypassing captchas: train the model to leverage this local representative information. To achieve this aim, we notice the masked autoencoders (MAE)-style self-supervised paradigm recently proposed by He *et al.* [25], which solves a similar problem in the image classification field. All the above motivates us to propose the basic idea of our method: “*destroy the captcha via a high masking ratio to force the encoder to learn the latent representation from local information (i.e., without the masking part)*”. Using the captcha with a high masking ratio as input, the encoder can sufficiently learn latent representation from local representative information to reconstruct the broken character. And then, we can build our powerful captcha decoder to break captchas based on the latent representation via semi-supervised learning, which avoids the reliance on a large number of labeled captchas. Note that we can obtain a powerful latent representation extractor only by leveraging unlabeled captchas.

Benefiting from the above idea, we propose GEESOLVER, a generic, efficient, and effortless solver based on self-supervised learning, to break text-based captchas. A total of two stages of training are involved in our solution. Specifically, in the first stage, we build an encoder and a reconstruction decoder based on Vision Transformer (ViT) [26]. Then, we use MAE-style paradigm to train our encoder for extracting latent representation from local information that can infer the corresponding character. After the first stage of training, the reconstruction decoder is discarded, while the ViT encoder is frozen and used as the latent representation extractor. Next, we develop a powerful captcha decoder, consisting of an information compression module, a sequence modeling module, and a decoupled attention module, for recognizing character sequences from learned latent representation. To take full advantage of unlabeled captchas, we train the captcha decoder with a semi-supervised method using a few labeled captchas and the same unlabeled captchas used in MAE paradigm. The latent representation extractor trained in the first stage and the captcha decoder trained in the second stage constitute the efficient model for text-based captcha recognition.

We evaluate the performance of GEESOLVER on widely used text-based captchas schemes. Experimental results show that our solver can more accurately recognize captchas than the state-of-the-arts with only 500 labeled captchas. Besides, we demonstrate that GEESOLVER is highly efficient as it can solve a captcha within 25 ms using a desktop CPU and 9 ms using a desktop GPU. In summary, the contributions of our paper are as follows:

- We develop an efficient solver for breaking text-based captchas, which consists of a ViT encoder and a captcha decoder. Our ViT-based encoder can map the observed captcha to the latent representation. The captcha decoder is built with an information compression module, a sequence modeling module, and a decoupled attention module. These three modules are well-designed according to the characteristics of the text-based captchas.
- We apply, for the first time, MAE-style self-supervised paradigm to build a latent representation extractor for captcha recognition. For reconstructing original captcha from little visible area, the latent representation extractor learns to extract latent representation from local information that can infer the whole character.
- We build our captcha decoder based on the latent representation via semi-supervised learning, which avoids the reliance on a large number of labeled captchas.
- We evaluate GEESOLVER on the current mainstream captcha schemes. Experimental results show that our solver can achieve excellent recognition performance with limited labeled captchas. Furthermore, GEESOLVER also has high attack success rates on the two previous captcha datasets, indicating that our solver outperforms the state-of-the-art methods by a large margin.

2. Background

2.1. Threat Model

Over the years, text-based captchas have been widely used to distinguish computer programs from humans. Recently, researchers have proposed some modern captcha systems that are different from the text-based captcha system in their interaction mode. For instance, image-based captchas ask the user to select all squares that satisfy the condition, which requires users to have some prior knowledge to select correctly. Behavior-based captchas (e.g., reCAPTCHA v3) perform risk analysis according to the critical steps of the user journey. The design of the risk analysis algorithm and the trade-off between the false positive rate and the recall rate also bring challenges for webmasters. Therefore, due to friendliness and reliability, text-based captchas are still used by many famous websites (e.g., Google, Yandex, and Microsoft) on user login pages. After entering incorrect passwords multiple times, the user must submit the results of text-based captchas. We collected eight text-based captcha schemes from the top 50 most popular websites ranked by *Alexa.com*.

In this work, we aim to design an efficient solver trained with a few labeled captchas and many unlabeled captchas to break text-based captchas with different security features effectively. In recent years, to resist various attack methods, experts have implemented many security features in text-based captchas to ensure website security, including character security features and background security features. These security features render segmentation-based approaches obsolete and bring challenges for building DL-based solvers with limited labeled captchas. Some work [21], [23], [24]

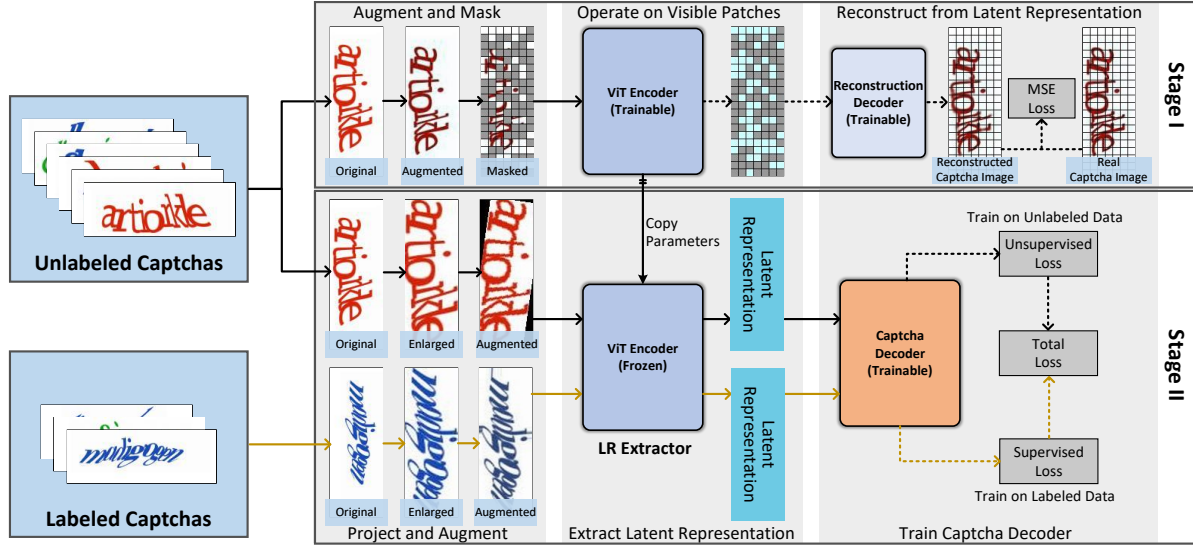


Figure 1. The schematic illustration of our approach for breaking text-based captchas. We first design a generic and efficient baseline model (GEE SOLVER model; see Figure 2) to break captchas with a ViT-based latent representation extractor and a captcha decoder. Then, in stage I, we leverage unlabeled captchas to train our latent representation extractor with the MAE-style paradigm. In stage II, the same unlabeled captchas and a few labeled captchas (additional) are used to train the captcha decoder with semi-supervised method.

has shown that it is difficult for solvers to achieve high attack success rates on three text-based captcha schemes (i.e., Google, Yandex, and Microsoft). Among them, the *Google* captcha and *Yandex* captcha introduce CCT technology, whose characters are connected to resist segmentation. Besides, the *Google* captcha additionally implements a high degree of rotation and distortion to make it unrecognizable. The degrees of rotation and distortion of *Yandex* captcha and *Microsoft* captcha are lower than those of *Google* captcha. However, they are designed with hollow fonts to increase the difficulty of extracting individual characters. Thus, we define these three captcha schemes as difficult text-based captcha schemes in our work. Obviously, once the security features of difficult captcha schemes are adopted by more captcha schemes, the effectiveness and generality of prior solvers will suffer.

We assume that attackers can quickly collect a large number of unlabeled captcha samples through distributed crawlers and label a small number of captcha samples at a low cost. Furthermore, the attackers also have the computing power to train the solver. Since the captcha schemes have different features, the solver for breaking each captcha scheme is trained by the corresponding captcha samples.

2.2. Design Goals

Many previous works have been devoted to designing efficient solvers for captcha recognition. However, they all suffer from one or more of the following problems. First, previous solvers have achieved significant attack success rates for simple captcha schemes, but they are still unable to break the captcha scheme with complicated security features (e.g., CCT). Second, while mainstream DL-based solvers outperform, they require a significant number of

labeled captchas for training. It requires a significant amount of time and effort to label captchas anew if the captcha style changes or new security features are implemented. Third, over-reliance on highly complex preprocessing and segmentation techniques makes the recognition process complicated and inefficient. In light of the challenges listed above, GEE SOLVER hopes to achieve the following goals:

Generic. The solver should adopt a generic method to recognize various captcha schemes with diverse security features, which is a prerequisite for the long-term availability of the captcha solver. In addition, the solver should have a high attack success rate since an unsuccessful attack may activate the related protective mechanism.

Efficient. The solver should be able to break captchas without relying on a sophisticated preprocessing mechanism in order to undertake real-time assaults. In other words, the entire assault procedure must be as quick as feasible.

Effortless. Since labeling captchas is a time-consuming and labor-intensive task, we need to propose a method to train our solver with a few labeled captchas. By introducing self-supervised and semi-supervised paradigms for the first time, we mitigate the reliance on labeled samples with the help of unlabeled samples.

Note that previous solutions mostly rely on obtaining a cleaner or simpler captcha image, which reduces the difficulty for the solver to recognize. The performance of these solvers degrades significantly as the captcha security features (especially for characters) become more complicated. Thus, we act in a diametrically opposite way to train the latent feature extractor of our solver. An extremely difficult-to-recognize captcha with a high masking ratio is used as input during training, so that the latent representation extractor learns latent representation only

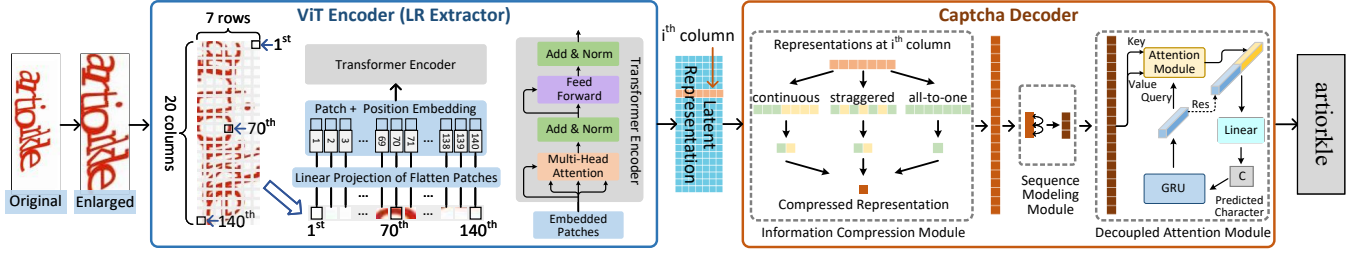


Figure 2. The schematic illustration of the proposed GeeSolver model in Figure 1, which is composed of a ViT encoder for latent representation extraction and a captcha decoder for captcha character recognition.

from local information to reconstruct the target captcha. Obviously, since the extractor can extract the useful latent representation for reconstruction in the above extreme cases, the captcha with security features has richer information, allowing the extractor to cope with it easily.

3. Methodology

In this section, we first provide an overview of our novel training paradigm using a few labeled captcha and many unlabeled captcha samples. Further, we describe how to build the latent representation extractor in Stage I and the captcha decoder in Stage II.

3.1. Overview of Our Approach

The overall framework of our approach is presented in Figure 1 and the training algorithm is illustrated in Appendix I. Our method consists of three components, a ViT [26] encoder for extracting the latent feature representation of captcha, a reconstruction decoder for facilitating the training of the ViT encoder, and a captcha decoder for recognizing captcha. The ViT encoder, whose parameters are frozen after the first stage of training, is a powerful latent representation extractor, which can provide a good representation for recognizing captcha. The latent representation extractor (i.e., the ViT encoder with frozen parameters) is trained in Stage I and captcha decoder is trained in Stage II, outlined as follows.

Stage I: Latent Representation Extractor. In Stage I, we introduce the MAE-style training paradigm [25], a recently proposed novel self-supervised learning method, to train the latent representation extractor with unlabeled samples. The key idea of MAE is to build a latent representation extractor with unlabeled data, which has achieved impressive results in the field of image classification. However, due to the newness of the technique, no work to date has yet exploited MAE-style paradigm to develop a generic solver for breaking captchas. To make the MAE paradigm more suitable for our breaking task, we adapt the structure of the ViT encoder and decoder, and divide a captcha image into 140 patches (i.e., $7rows \times 20columns$) according to the characteristics

of the text-based captcha. In addition, we also design a set of captcha augmentation methods to increase the task difficulty for more effective training. To be specific, we first split the captcha image into a sequence of non-overlapping patches. These patches are randomly masked and the ViT encoder extracts latent representation of only visible patches. Then the reconstruction decoder with small parameters attempts to reconstruct missing patches from visible latent representation. In order to reconstruct original captcha from little visible area, the ViT encoder has to extract latent representation from local information that can infer the whole character. After training, the ViT encoder is frozen and used as the powerful latent representation extractor for Stage II, while the ViT decoder is discarded.

Stage II: Captcha Decoder. Our captcha decoder is designed with an information compression module, a sequence modeling module, and a decoupled attention module. The captcha decoder can decode character sequences from learned representation effectively. To take full advantage of unlabeled samples, we use a few labeled captcha samples and many unlabeled captcha samples (i.e., same as used in Stage I) to train the captcha decoder with FixMatch [27], which is a state-of-the-art semi-supervised framework. Before training, we leverage a straightforward and efficient projection method to enlarge the text area to improve recognition performance further.

3.2. Stage I: Latent Representation Extractor

3.2.1. Structure of ViT Encoder. The ViT encoder, composed of a series of transformer blocks [28], can extract latent representation of image patches. Thus, it is also denoted as the latent representation extractor. The proposal of ViT has greatly promoted the further development of computer vision tasks [26], however, no previous work has considered using it to extract captcha features. As the first ViT-based encoder to extract the latent representation of the captcha image, the encoder structure is shown in Figure 2. Specifically, our encoder first divides a captcha image into non-overlapping patches of $7rows$ and $20columns$. Then ViT encoder embeds these flattened 140 patches by a linear projection with added positional embeddings. The resulting 140 embeddings are processed via a series of

transformer blocks. A transformer block consists of a multi-head attention sub-layer and a feed-forward sub-layer. Both two sub-layers use residual connections [29] followed by layer normalization [30]. After processing embeddings with several stacked transformer blocks, the ViT encoder extracts latent representation vectors of corresponding patches. In Stage I, some patches are randomly masked according to the masking rate and the ViT encoder extracts representation vectors of only visible patches. In Stage II, all patches are visible, and the ViT encoder extracts 140 representation vectors.

3.2.2. Structure of Reconstruction Decoder. The reconstruction decoder is used to facilitate the training of the ViT encoder by accomplishing the reconstruction task together with the ViT encoder. It consists of a few stacked transformer blocks. Given the representation vectors extracted from randomly masked patches, the reconstruction decoder first fills mask tokens at the position corresponding to the missing patches and gets 140 input embeddings. Similarly, the reconstruction decoder adds position embeddings and processes them with transformer blocks. Finally, the reconstruction decoder reconstructs all 140 patches at the pixel level. We compute reconstruction loss only on masked patches.

3.2.3. Train Latent Representation Extractor. We train our latent representation extractor with the MAE-style self-supervised training paradigm, which consists of the following five steps.

Step 1. Captcha Augmentation. As pointed out in [31]–[33], data augmentation methods play a key role in improving the quality of the learned visual representation. A total of 14 data augmentation methods are adopted (color, stretch, distort, etc.), as detailed in Appendix II. For a single captcha image, we randomly employ one augmentation method to it.

Step 2. Randomly Mask Patches. The input captcha $x \in \mathbb{R}^{H \times W \times C}$ is first split into a sequence of non-overlapping patches $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (H, W) is the resolution of the captcha image, C is the channel of the captcha image, (P, P) is the resolution of the image patch, and $N = H \cdot W / P^2$ is the number of patches. These patches are mapped to D -dimension vectors with a linear projection (i.e., a fully connected layer without bias), and then added with position embeddings, that is $x_{all} \in \mathbb{R}^{N \times D} = x_p W_b + E_{pos}$, where $W_b \in \mathbb{R}^{(P^2 \cdot C) \times D}$ is the weight of the fully connected layer, and $E_{pos} \in \mathbb{R}^{N \times D}$ is position embeddings. According to masking ratio α , $\alpha \times N$ patches in x_{all} are randomly masked, while the remaining patches $x_{vis} \in \mathbb{R}^{N' \times D}$ are still visible, where $N' = (1 - \alpha) \times N$ is the number of visible patches.

Step 3. Extract Latent Representation of Visible Patches. Taking x_{vis} as input, the ViT encoder extracts latent representation of only visible patches. The encoder is composed of a series of Transformer blocks. Transformer Block allows all input patches to interact without changing their shape. After processing x_{vis} via these Transformer

blocks, the latent representation of visible patches $z_{vis} \in \mathbb{R}^{N' \times D}$ are extracted.

Step 4. Reconstruct Missing Patches. Taking latent representation of visible patches z_{vis} as input, the ViT decoder reconstructs missing patches at the pixel level. The ViT decoder first fills $N - N'$ mask tokens at the position corresponding to the missing patches in z_{vis} , and then obtains the input $z_{all} \in \mathbb{R}^{N \times D}$. The mask token is a shared, trainable vector that indicates the patch for this location is masked. The decoder also adds trainable position embeddings to all patches: $z'_{all} \in \mathbb{R}^{N \times D} = z_{all} + E'_{pos}$, where $E'_{pos} \in \mathbb{R}^{N \times D}$ is another trainable position embeddings. Processing z'_{all} via several Transformer blocks in decoder and then mapping them into $(P^2 \cdot C)$ dimension vectors through a fully connected layer, pixel values for all patches $y_{all} \in \mathbb{R}^{N \times (P^2 \cdot C)}$ are reconstructed. Only reconstructed pixel values for missing patches $y_{mis} \in \mathbb{R}^{(N - N') \times (P^2 \cdot C)}$ are leveraged to compute reconstruction loss.

Step 5. Compute Reconstruction Loss. Reconstructed pixel values for missing patches $y_{mis} \in \mathbb{R}^{(N - N') \times (P^2 \cdot C)}$ can be easily reshaped to form reconstructed image patches $y_{rec} \in \mathbb{R}^{(N - N') \times P \times P \times C}$. Then the mean squared error (MSE) between reconstructed patches y_{rec} and original patches $y_{real} \in \mathbb{R}^{(N - N') \times P \times P \times C}$ is computed as reconstruction loss:

$$\mathcal{L}_{rec} = MSE(y_{rec}, y_{real}). \quad (1)$$

After training, the ViT encoder is able to extract latent representation of captcha image patches. Then, its parameters are frozen, and we use it as the latent representation extractor for Stage II.

3.3. Stage II: Captcha Decoder

3.3.1. Structure of Captcha Decoder. According to the common characteristics of captcha images (i.e., the text is arranged horizontally from left to right), we design the captcha decoder composed of an information compression module, a sequence modeling module, and a decoupled attention module, to effectively decode character sequences from learned latent representation.

Information Compression Module. Given a captcha image $x \in \mathbb{R}^{H \times W \times C}$, the frozen latent representation extractor trained in Stage I extracts latent representation vectors of all patches $z \in \mathbb{R}^{N \times D}$, where $N = H \cdot W / P^2$ is the number of patches and D is the dimension of latent representation vectors. These latent representation vectors z are then reshaped to $z_{row \times column} \in \mathbb{R}^{\frac{H}{P} \times \frac{W}{P} \times D}$ according to the position of patches in the origin image. Compressing representation vectors of patches in the same column can reduce the input dimension without affecting the sequential representation. We design three information compression modules to achieve this, as shown in Figure 2. (a) Continuous compression: for representation vectors at i -th column $z_i \in \mathbb{R}^{\frac{H}{P} \times D}$, z_i is divided into two groups z_i^a and z_i^b in a continuous

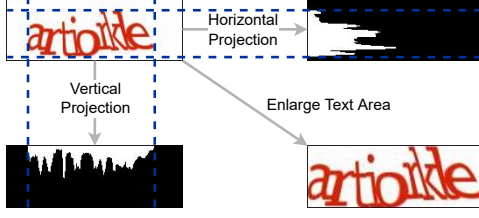


Figure 3. The process of enlarging text area in captcha image by horizontal projection and vertical projection.

manner, where $z_i^a, z_i^b \in \mathbb{R}^{\frac{H}{2P} \times D}$. Then average pooling is leveraged to compress two groups of representation vectors: $\bar{z}_i^a = \text{AvgPool}(z_i^a), \bar{z}_i^b = \text{AvgPool}(z_i^b), \bar{z}_i^a, \bar{z}_i^b \in \mathbb{R}^D$. Finally, two average vectors are concatenated as compressed representation vector at i-th column: $c_i = \text{Concat}(\bar{z}_i^a, \bar{z}_i^b) \in \mathbb{R}^{2D}$; (b) Staggered compression: z_i is divided into two groups in a staggered manner, the other processing steps are the same as continuous compression; (c) All-to-one compression: average all representation vectors in z_i directly with average pooling, and get compressed representation vector at i-th column: $c_i \in \mathbb{R}^D$.

The information compression module compresses the information of each column, and generates the final compressed representation vectors for all columns $c \in \mathbb{R}^{\frac{W}{P} \times 2D}$ (continuous and staggered compression) or $c \in \mathbb{R}^{\frac{W}{P} \times D}$ (all-to-one compression), representing the sequence information of captcha from left to right.

Sequence Modeling Module. Each representation vector in c is a compressed representation of one column. However, these representation vectors suffer the lack of contextual information [34]. Therefore, we use the sequence modeling module to allow adjacent representation vectors to exchange information. The sequence modeling module is actually a two-layer GRU [35]:

$$s = \text{GRU}(c) \quad s \in \mathbb{R}^{\frac{W}{P} \times D'}, \quad (2)$$

where s is the contextual compressed representation vectors and D' is the hidden size of GRU.

Decoupled Attention Module. A sequence of characters is predicted by the decoupled attention module from the contextual compressed representation vectors s step by step. The architecture of the decoupled attention module is illustrated in Figure 2. At time step t , another two-layer GRU generates the *query* vector:

$$q_t = \text{GRU}(y_{t-1}, h_{t-1}), \quad (3)$$

where y_{t-1} is the detached one-hot prediction at time step $t-1$, h_{t-1} is the hidden state of GRU at time step $t-1$, and q_t is considered the *query* vector at time step t . Then the attention map is computed:

$$\alpha_i^t = \text{softmax}(q_t^T s_i), i = 1, \dots, \frac{W}{P}, \quad (4)$$

where q_t^T indicates the transpose of vector q_t , and s_i represents i-th contextual compressed vector. Then the

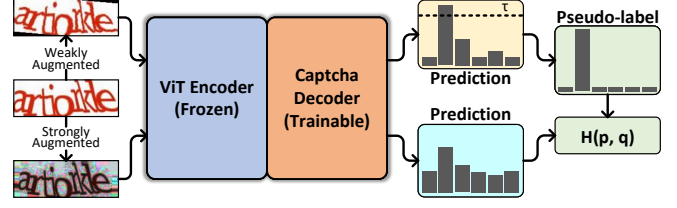


Figure 4. Training process of captcha decoder with FixMatch.

weighted aggregation of contextual compressed vector s is computed:

$$\mu_t = \text{Concat}\left(\sum_{i=1}^{\frac{W}{P}} \alpha_i^t s_i, q_t\right), \quad (5)$$

where we use a residual connection to concatenate q_t into the output vector. After that, we use a fully connected layer to obtain the output prediction at time step t :

$$c_t = \text{softmax}(\text{FC}(\mu_t)), \quad (6)$$

then we obtain the one-hot prediction at time step t , denoted as y_t .

3.3.2. Enlarge Text Area. Before training the captcha decoder, we leverage standard projection method [36] and design anti-occluding line projection method to enlarge the text area for all labeled and unlabeled samples. The process of standard projection method is illustrated in Figure 3. First, we convert the input captcha image into binary image using standard threshold method [37]. Then we use horizontal projection and vertical projection to locate and extract the text area from the binary image. For horizontal projection, we scan the binary image from the top to the bottom and obtain horizontal histogram. In horizontal histogram, the value of a histogram bin is the sum of the non-zero pixels along a particular line in horizontal direction. We delete the regions with value 0 on both sides of the image. For vertical projection, we scan the binary image from the left to the right and obtain vertical histogram likewise. After removing extraneous areas in horizontal and vertical directions, we resize the text area to the same size as the original captcha image. To avoid occluding lines invalidating our projection method, we design an anti-occluding line mechanism to overcome this commonly used security feature. Instead of the sum of the non-zero pixels, the value of a histogram bin is the number of consecutive occurrences of non-zero pixels. Regions with value 0 are blank areas, regions with value 1 are considered areas with occluding lines, and only areas with a value of greater than 1 will be reserved. By enlarging text area, we make the patches of the same column as little as possible to contain two adjacent characters.

3.3.3. Train Captcha Decoder with FixMatch. To take full advantage of unlabeled data, we use FixMatch [27], a state-of-the-art semi-supervised framework to train the captcha decoder with a few labeled samples and the same unlabeled captcha samples used in Stage I. In addition to

supervised loss, the captcha decoder is required to keep consistent predictions for strongly augmented data and weakly augmented data of the same unlabeled images. The training process of FixMatch is shown in Figure 4.

The supervised loss is computed through cross-entropy loss:

$$\mathcal{L}^s = H(f(x), y), \quad (7)$$

where x is labeled sample, $f(x)$ is prediction on x by the frozen latent representation extractor and the captcha decoder, and y is the corresponding label.

For an unlabeled sample x_u , we first apply different augmentation methods to obtain weakly augmented sample x_u^W and strongly augmented sample x_u^S . Then we compute the prediction result on x_u^W : $q = f(x_u^W)$. The prediction result with high confidence is taken as the pseudo-label $\hat{q} = \operatorname{argmax}(q_n)$. The unsupervised loss is computed as:

$$\mathcal{L}^u = \mathbb{1}(\max(q) \geq \tau) H(f(x_u^S), \hat{q}), \quad (8)$$

where τ is the confidence threshold. FixMatch combines the supervised loss $\mathcal{L}^s$ and the unsupervised loss $\mathcal{L}^u$ to train the captcha decoder.

4. Experimental Results

4.1. Experiment Settings

We evaluate our approach on eight text-based captcha schemes of the top-50 most popular websites ranked by Alexa.com. For each captcha scheme, we construct a dataset consisting of three subsets: a labeled subset, an unlabeled subset, and a test subset. The unlabeled subset is used to train the latent representation extractor, and the labeled and unlabeled subsets are combined to train the captcha decoder. The test subset is used to assess our solver’s performance. First, we collected 7,000 captcha images from each target website. Two thousand of them are manually labeled. The training subset consists of 500 labeled samples, the test subset consists of 1,500 labeled samples, and the unlabeled subset consists of 5,000 unlabeled samples. Details are in Appendix III. In comparison to previous attack methods, we expand our evaluation to ten different captcha schemes taken from two datasets utilized in previous research and rigorously follow their evaluation process. We include implementation details in Appendix IV.

4.2. Evaluation on Current Captcha Schemes

To prove the effectiveness of GEESOLVER, we evaluate it on eight text-based captcha schemes. For each scheme, we train GEESOLVER with 500 labeled and 5,000 unlabeled samples. The training time on each captcha scheme is about ten hours using an NVIDIA GeForce RTX3090 GPU. TABLE 1 shows the accuracy and the average recognition time for solving a captcha.

GEESOLVER has achieved success rates ranging from 90.73% to 99.73% on eight captcha schemes, with all of them over 90 percent. Although these eight captcha schemes

TABLE 1. THE ACCURACY AND THE RECOGNITION TIME PER CAPTCHA FOR EIGHT TEXT-BASED CAPTCHA SCHEMES. We deploy GEESOLVER on a lower performance PC with Intel® Core™ i7-10700@2.90 GHz, 16 GB RAM, and an NVIDIA GeForce RTX3060 GPU. **RT-CPU** is the average recognition time tested on the CPU mode and **RT-GPU** is tested on the GPU mode.

Scheme	Example	Accuracy	RT-CPU	RT-GPU
Google		90.73%	24.47 ms	8.21 ms
Yandex		92.87%	23.99 ms	7.57 ms
Microsoft		97.41%	24.98 ms	7.87 ms
Wikipedia		97.80%	24.65 ms	8.24 ms
Weibo		92.47%	22.45 ms	6.39 ms
Sina		97.00%	22.72 ms	6.89 ms
Apple		95.60%	23.45 ms	7.06 ms
Ganji		99.73%	22.70 ms	6.60 ms

employ various security features to increase recognition difficulty, our solver can still recognize them accurately. Among them, the highest success rate of 99.73% is achieved on the *Ganji* captcha scheme, which has simpler security features leading to easier breaking. It is worth noting that the accuracy remains quite good even on three difficult captcha schemes (i.e., *Google*, *Yandex*, *Microsoft*) using complicated security features. We also train GEESOLVER with all collected captcha samples for generality evaluation. The accuracy of training on all collected captcha samples is lower than that of the specific captcha scheme, but the solver still has good performance (see Appendix V for detailed analysis). Besides, GEESOLVER can recognize a captcha image of any scheme within 25 ms in the CPU mode and within 9 ms in the GPU mode. Thus, it could be concluded that GEESOLVER has an extremely fast recognition speed and can perform real-time attacks on text-based captchas.

4.3. Comparison with Prior Work

We first compare GEESOLVER to four state-of-the-art attacks on 5 captcha schemes in *dataset A* (collected by Tang *et al.* [16]) and 5 captcha schemes in *dataset B* (collected by Ye *et al.* [23]), respectively. To ensure a fair comparison, we perform the same attack tasks with the training and test datasets used in [16], [21], [23], [24]. We carefully follow their evaluation protocol, as detailed in Appendix VI. Note that we train GEESOLVER for breaking each captcha scheme using the corresponding captcha samples, as in previous work. There are two works that also aim to implement solvers with a small amount of labeled captchas, one by Ye *et al.* [23] (500 labeled captchas for *dataset B*) and the other by Tian *et al.* [24] (2,000 labeled captchas for *dataset A* and 500 labeled captchas for *dataset B*). The other two works train the solver with the conventional supervised learning paradigm, where

TABLE 2. PERFORMANCE COMPARISON BETWEEN OUR SOLVER WITH FOUR STATE-OF-THE-ART ATTACKS. We leverage only 500 labeled samples and additional 1,000 or 5,000 unlabeled samples to train our solver for each captcha scheme.

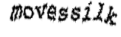
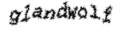
Dataset A	Example	Ref. [16]	Ref. [24]	Ref. [21]	GEESOLVER		Dataset B	Example	Ref. [23]	Ref. [24]	GEESOLVER	
					1000	5000					1000	5000
Yandex		56.0%	63.2%	74.6%	74.10%	76.18%	Google		3.0%	9.0%	52.26%	82.92%
Microsoft		50.9%	56.6%	59.8%	22.67%	74.76%	Microsoft		69.6%	75.5%	78.50%	86.00%
Sina		75.0%	84.0%	74.1%	88.60%	88.93%	Sina		52.6%	63.5%	80.50%	81.50%
Weibo		51.2%	88.2%	88.4%	93.98%	94.38%	Weibo		44.0%	82.5%	93.50%	99.50%
Wikipedia		90.0%	87.0%	88.8%	91.03%	91.31%	Wikipedia		78.0%	66.5%	86.22%	87.25%

TABLE 3. THE COMPARISON RESULTS OF REF. [38] AND GEE SOLVER ON OUR DATASET USING 500 LABELED SAMPLES AND 5000 UNLABELED SAMPLES.

Scheme	Ref. [38]	Ours	Scheme	Ref. [38]	Ours
Google	76.4%	90.7%	Weibo	92.5%	92.5%
Yandex	90.4%	92.9%	Sina	97.1%	97.0%
Microsoft	96.4%	97.4%	Apple	92.9%	95.6%
Wikipedia	98.5%	97.8%	Ganji	99.4%	99.7%

Tang *et al.* [16] used 2,000 labeled captchas and Zi *et al.* [21] used 6,000 labeled captchas. To be more in line with the motivation of our work, we leverage only 500 labeled captchas for both datasets and use additional 1,000 or 5,000 unlabeled captchas to train our solver. It can be seen from TABLE 2 that GEESOLVER outperforms four state-of-the-arts by a large margin with the same or even less number of labeled captchas, especially our solver achieving an accuracy of 82.92% on *Google* captchas in *dataset B*. In the same case of using 500 labeled captchas, Ye *et al.* [23] and Tian *et al.* [24] reported 3% and 9% accuracy on *Google* captchas, respectively. Besides, we note that the improved FixMatch semi-supervised framework proposed by Deng *et al.* [38] effectively improves the performance of the solver with unlabeled captcha samples. Based on their open source-code, we conduct experiments on the same captcha samples for fair comparison. TABLE 3 shows that the performance of GEESOLVER on the hard-to-attack captcha schemes is still significantly better than their solver. In addition, we believe that there is a “ceiling effect”, i.e., the performance of different methods will be similar when the captcha is simple or the training data is sufficient. Thus, we perform experiments using fewer labeled training captcha samples for comparison. As shown in Figure 5, the success rates of our method are still very high in the case of 200 labeled samples, while their method performs very poorly under the same condition. It can be concluded that GEESOLVER has successfully broken various text-based captcha schemes and achieved a significant breakthrough in solving the difficult captcha schemes with complicated security features.

The key reason for the apparent performance improvement is our unique insights into the characteristics of text-

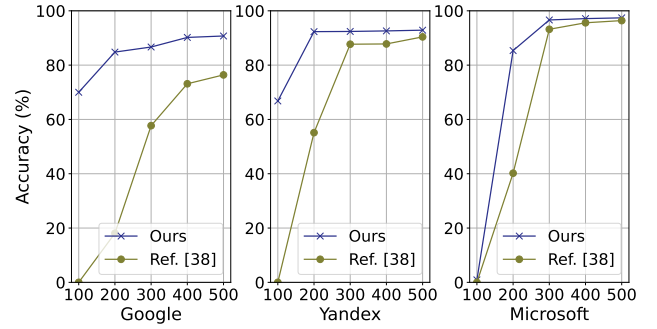


Figure 5. Performance comparison between GEESOLVER with Ref. [38] on three difficult captcha schemes using different numbers of labeled samples.

based captchas, based on which we carefully design our training paradigm and solver model. Our solution leverages the “destroyed” captcha with a high masking rate as input to train the encoder to learn latent representation from local information, which contrasts with previous solvers that mostly use preprocessed cleaner and simpler captcha for training and recognition. Obviously, this extreme case of masking is more complicated than any captcha security feature, and it also facilitates our encoder to extract a more effective representation when solving difficult captchas. Furthermore, the semi-supervised training paradigm further leverages unlabeled captchas, and the superior design of our model also guarantees the effectiveness of learning.

4.4. Main Properties of LR Extractor

In Stage I, we build the latent representation extractor (LR extractor) using the MAE-style training paradigm. The quality of the latent representation is essential for the downstream task (i.e., captcha recognition). In this section, we first demonstrate the value of the MAE paradigm for training the latent representation extractor by conducting experiments to directly train the GEESOLVER model with the ViT encoder not trained by the MAE paradigm. Then, to demonstrate the high quality of the latent representation, we exhibit the reconstructed captcha. Finally, we explore

TABLE 4. EXPERIMENTAL RESULTS OF TRAINING OUR SOLVER WITHOUT USING MAE. Conv.-Itera. represents the number of iterations when the highest accuracy is obtained. A value of “n.a.” indicates that the training failed to converge.

Scheme	Ours		w/o MAE	
	Acc.	Conv.-Itera.	Acc.	Conv.-Itera.
Google	90.73%	70k	n.a.	n.a.
Yandex	92.87%	16k	n.a.	n.a.
Microsoft	97.41%	65k	n.a.	n.a.
Wikipedia	97.80%	6k	96.53%	50k
Weibo	92.47%	4k	88.33%	100k
Sina	97.00%	6k	95.53%	65k
Apple	95.60%	20k	64.86%	100k
Ganji	99.73%	10k	95.73%	75k

the influence of the masking ratio on the quality of learned representation.

4.4.1. MAE-Style Paradigm Contribution. To prove the importance of pre-training latent representation extractor with MAE in Stage I, we do ablation experiments by directly training the solver model with FixMatch. The experimental results are shown in TABLE 4. According to the results, without the representation learned by MAE, the solver fails to recognize three difficult captcha schemes due to the small number of labeled samples. These three difficult captcha schemes employ complex security features to destroy the standard characters, which brings noise and variation to the characters. MAE forces the latent representation extractor to learn key local representative information rather than noise. From the key local representative information, the high-quality representation is extracted and the whole character can be easily inferred from it, which is the key reason why GEESOLVER performs so well on complex captcha schemes. For other simple captcha schemes, the solver training without the MAE-style paradigm also shows lower recognition performance and much longer training time. In contrast, training a captcha decoder based on latent representation is very fast. Furthermore, we ablate FixMatch training method in the second stage to further observe the contribution of latent representation. We train the captcha decoder based on the latent representation with supervised learning (i.e., without FixMatch in TABLE 5). It can be seen that, with the help of high-quality representation, the solver achieves over 90% accuracy for simple captcha schemes. For difficult captcha schemes (i.e., Google, Yandex, and Microsoft), the accuracy is 72.33%, 85.00%, and 55.67%, respectively. Thus, it can be concluded that the use of the latent representation extractor is the key to successfully breaking difficult captchas.

4.4.2. Reconstruction Performance. In order to prove that ViT encoder has learned to extract useful representation that can infer the whole character, we exhibit the reconstruction performance of MAE, as shown in Figure 6. All examples are selected from *test* set and they are not used in MAE training. The results show that even in the case of such

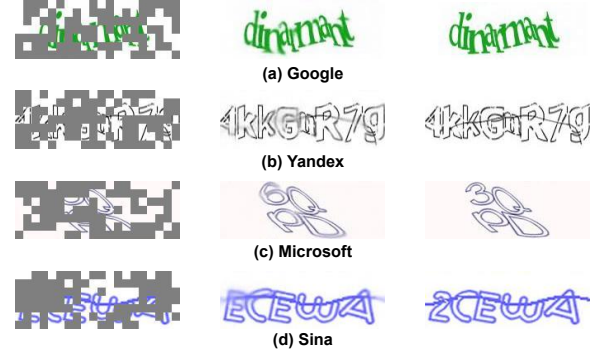


Figure 6. Reconstruction results on *test* set. For each triplet, we exhibit the masked captcha (left), MAE reconstruction (middle), and the ground-truth (right). More examples are in the Appendix VII.

a high masking ratio (60%), MAE can still reconstruct high-quality and correct captcha images from a few visible patches. The key reason is that the entire character can be speculated through a small visible part of the character. An intriguing phenomenon is that MAE sometimes infers missing patches to produce different yet plausible captcha images. For example, MAE reconstructs the Microsoft captcha with the ground truth of “3Q-PD” to “6Q-PD”, and reconstructs the Sina captcha with the ground truth of “2CEwA” to “ECEwA”. Through the small visible part, there may be a variety of reasonable reconstruction for this character. This reasoning-like behavior indicates that the ViT encoder can extract high-quality latent representation.

4.4.3. Impact of Masking Ratio. To explore the influence of masking ratio on the quality of learned representation, we select three difficult captcha schemes for ablation experiments. Figure 7 shows the influence of the masking ratio. Initially, with the increase in masking ratio, the accuracy also increases, and the optimal masking ratio for all three captcha schemes is 60%. However, when the masking ratio exceeds 60%, making reconstructing missing patches becomes an over-difficult task, and the accuracy decreases rapidly. We note that the optimal masking ratio in [25] is 75% for natural images. The reason is that each character in the captcha image is independent, which makes the information density of the captcha image slightly higher than that of the natural image. We also analyze the impact of captcha image augmentation (details are in Appendix VIII).

4.5. Main Properties of Captcha Decoder

In Stage II, we design a captcha decoder composed of an information compression module, a sequence modeling module, and a decoupled attention module. Before training, we enlarge the text area for all captcha images and use FixMatch to train the captcha decoder. In this section, we first do ablation experiments on model structure (i.e., three modules of our captcha decoder). Then, we illustrate the importance of the training strategy (i.e., FixMatch).

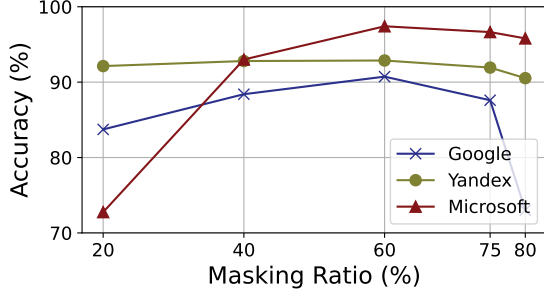


Figure 7. The influence of masking ratio on the quality of learned representation.

TABLE 5. ABLATION STUDY FOR CAPTCHA DECODER ON MODEL STRUCTURE AND TRAINING STRATEGY. The abbreviations are explained as follows, **w/ M-I**: only with staggered compression module, **w/ M-II**: only with all-to-one compression module, **w/o M-III**: without sequence modeling, **T-A**: using traditional attention mechanism, and **w/o FM**: without FixMatch.

Scheme	Ours	Model Structure				Training
		w/ M-I	w/ M-II	w/o M-III	T-A	w/o FM
Google	90.73%	87.93%	86.20%	54.00%	87.80%	72.33%
Yandex	92.87%	92.00%	91.93%	49.87%	90.67%	85.00%
Microsoft	97.41%	74.23%	69.54%	74.37%	94.40%	55.67%
Wikipedia	97.80%	97.93%	97.60%	95.80%	97.33%	95.20%
Weibo	92.47%	92.47%	92.47%	92.40%	92.47%	92.07%
Sina	97.00%	97.00%	96.93%	95.53%	97.00%	96.87%
Apple	95.60%	95.26%	95.06%	83.68%	91.06%	91.06%
Ganji	99.73%	99.60%	99.40%	98.60%	99.33%	98.40%

Finally, we show the benefits of enlarging the text area for recognizing captchas with complex security features.

4.5.1. Model Structure. According to the common characteristics of captcha images, we build the captcha decoder with an information compression module, a sequence modeling module, and a decoupled attention model.

Information Compression Module. We design three information compression modules and employ continuous compression module by default. The comparison results with the other two compression modules are shown in TABLE 5. For most schemes, continuous compression module is the best choice, and staggered compression module is slightly worse than continuous compression module. All-to-one compression module retains the least information resulting in the least effectiveness. Grouping representation vectors of a column in a continuous manner maximize the preservation of character and sequence information. Besides, for *Microsoft* captchas, which employ a security feature of the two-layer structure, continuous compression module is perfect for preserving information on characters in upper-layer and lower-layer, respectively. Compared to 74.23% and 69.54% of the other two modules, continuous compression module obtains an accuracy of 97.41%.

Sequence Modeling Module. We use sequence modeling module to allow adjacent compressed vectors by column to exchange information. Ablation experimental results

TABLE 6. THREE ERROR CATEGORIES FOR PREDICTING GOOGLE CAPTCHAS WITHOUT USING SEQUENCE MODELING MODULE.

Scheme	Error Category	Captcha Image	Ground Truth	Solver Output
Google	Missing Prediction		ansabsgic	ansabsgi
			anteducks	anteduck
	Wrong Prediction		alldingsi	allingsi
			ballessi	ballessi
	Duplicate Prediction		anctinsf	anctinsff
			bastbags	bastbags

on sequence modeling module are shown in TABLE 5. Removing sequence modeling module leads to a decline in accuracy in different degrees for all captcha schemes. Among them, the accuracy of *Google* captchas drops from 90.73% to 54.00%, and the accuracy of *Yandex* captchas drops from 92.87% to 49.87%. These two captcha schemes employ the security feature of CCT. Since there is no blank area between adjacent characters, it is hard for captcha decoder to judge which columns a single character is distributed in without using sequence modeling module. Decoding character sequences from compressed vectors by column without knowing their contextual information will result in missing predictions, wrong predictions, or duplicate predictions. Some error examples of *Google* captchas are listed in TABLE 6. All these samples can be correctly recognized by GEESOLVER with sequence modeling module.

Decoupled Attention Module. Unlike traditional attention-based decoders, who struggle to align long sequence [39], the decoupled attention module decouples *query* vector from representation vectors. Comparison results are shown in TABLE 5. For Wikipedia, Weibo, Sina, and Ganji captchas with only simple security features, such as occluding lines, the accuracy slightly drops. However, for captcha schemes that employ more complicated security features, the accuracy drops 2% to 5% (Google: 2.93%, Microsoft: 3.01%, Yandex: 2.20%, Apple: 4.54%). This indicates that decoupled attention mechanism performs better when trained on small amounts of data. Moreover, decoupled attention module can compute outputs of all steps in parallel when using teacher forcing strategy, which reduces the training time.

4.5.2. Training Strategy (FixMatch). In this experiment, we freeze the ViT encoder and leverage a few labeled captchas to train captcha decoder with supervised learning (i.e., without FixMatch). As we analyzed in Section 4.4.1, benefiting from latent feature extraction, the solver can also achieve 72.33%, 85.00% and 55.67% accuracy for the three difficult captcha schemes (i.e., Google, Yandex, and Microsoft), respectively. To take full advantage of unlabeled

TABLE 7. THE PERFORMANCE AND INFLUENCE OF PROJECTION METHODS FOR ENLARGING THE TEXT AREA.

Scheme	Example		Accuracy	
	w/o Proj.	with Proj.	w/o Proj.	with Proj.
Google			77.20%	90.73%
Yandex			92.87%	93.87%
Microsoft			97.41%	97.41%
Wikipedia			97.80%	97.80%
Weibo			92.47%	92.47%
Sina			97.40%	97.40%
Apple			96.48%	96.48%
Ganji			99.67%	99.73%

TABLE 8. STATISTICS ON THE INCORRECTLY RECOGNIZED CAPTCHA SAMPLES ABOUT THE EDIT DISTANCE (ED) AND ERROR CATEGORY. The abbreviations are explained as follows, **A**: missing prediction, **B**: wrong prediction, **C**: duplicated prediction, **FR**: failure rate.

Scheme	ED=1			ED=2			ED=3			ED Ratio			FR
	A	B	C	A	B	C	A	B	C	1	2	3	
Google	14	90	5	10	27	7	5	16	3	78%	16%	6%	9.3%
Yandex	5	82	8	2	14	4	1	4	1	89%	9%	2%	7.1%
Microsoft	7	29	0	0	5	1	0	0	0	92%	8%	0%	2.6%
Wikipedia	0	30	2	0	2	0	0	0	0	97%	3%	0%	2.2%
Weibo	0	107	0	0	12	0	0	0	0	95%	5%	0%	7.5%
Sina	0	44	0	0	2	0	0	0	0	98%	2%	0%	3.0%
Apple	6	54	0	1	11	0	0	0	0	91%	9%	0%	4.4%
Ganji	0	4	0	0	0	0	0	0	0	100%	0%	0%	0.3%

data, we leverage FixMatch to train the captcha decoder with a few labeled samples and the same unlabeled captchas used in Stage I. It can be seen from TABLE 5, with the help of unlabeled samples leveraged by FixMatch, the accuracy for three difficult captcha schemes increases to 90.73%, 92.87%, and 97.41%. For common image classification tasks, they directly use the normal full-supervised method to train the decoder after training the backbone with self-supervised learning [25], [40], [41]. This is because the decoder used in image classification tasks is simple, usually a fully connected layer. However, for the captcha recognition task, the decoder structure is complicated, and directly training it with full-supervised learning method leads to varying degrees of overfitting.

4.5.3. Enlarge Text Area. In Stage II, we leverage the projection method to enlarge the text area before training the captcha decoder. The performance and influence of projection methods are shown in TABLE 7. The projection methods are very effective for Google and Yandex captchas and improve the accuracy from 77.20% and 92.87% to 90.73% and 93.87%, respectively. For other captchas, it seems that enlarging the text area has a limited effect on

TABLE 9. WRONG PREDICTIONS BY GEESOLVER.

Scheme	Captcha Image	Ground Truth	Solver Output
Google		metenessfu	metenesstu
Microsoft		5W-4LS	5W-4SS
Sina		6FWsW	6FWsV
Apple		U3J7W	UBJ7W
Ganji		TCnL	T6nL

helping the captcha decoder recognize more accurately. Both Google and Yandex captchas have characters that occupy very small space and stick very close together, so enlarging the text area can effectively improve recognition accuracy.

4.6. Incorrectly Recognized Samples

To explore the weaknesses of our approach, we list the statistics on the error prediction samples about the edit distance and error category, as shown in TABLE 8. It is worth noting that the edit distance between incorrect predictions and ground truth is 1 in most instances, which indicates that incorrect predictions are very close to correct answers. Simply inserting, replacing, or deleting one character in the mispredicted sequence will get the correct answer. Among the three error categories, the most common is the wrong prediction. To be more intuitive, we exhibit some wrong predictions by GEESOLVER in TABLE 9. Our solver mispredicts one character due to several reasons:

- 1) When two adjacent characters are distributed in the same column, e.g. “LS” in Microsoft captcha, the solver is likely to predict the same character twice from these columns.
- 2) The character and occluding lines form a plausible character, e.g. “C” in Ganji captcha.
- 3) The character and a part of its adjacent character form a plausible character, e.g. “3” in Apple captcha.
- 4) The character is distorted or deformed so much that it looks like another character, e.g. “f” in Google captcha.
- 5) A part of the character does not appear in the image, the rest looks like another character, e.g. “W” in Sina captcha.

4.7. Impact of Labeled and Unlabeled Data Sizes

Because labeling captchas is usually time-consuming, we investigate the trade-off between accuracy and the number of labeled samples. Figure 5 shows that there is a dividing line between 100 and 200 labeled samples for GeeSolver. Beyond this line, there is no significant improvement in accuracy. However, once it is below this dividing line, the accuracy will drop dramatically, even approaching 0% for Microsoft captchas. Besides, unlabeled samples are critical for training GEESOLVER because they are leveraged with both self-supervised learning in Stage I

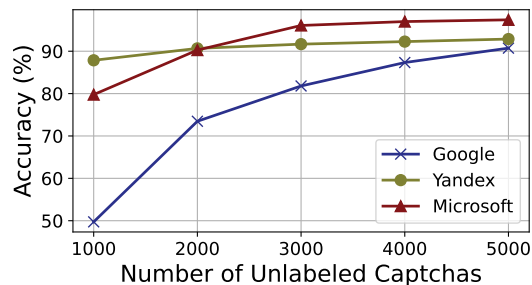


Figure 8. The success rates of GEESOLVER on three difficult captcha schemes when using different numbers of unlabeled samples.

and semi-supervised learning in Stage II. Figure 8 shows the success rates of GEESOLVER on three difficult captcha schemes when using a different number of unlabeled samples. The results of comparative experiments have demonstrated that GEESOLVER does not rely on a large number of unlabeled samples for recognizing simple captcha schemes. However, for three difficult captcha schemes, more unlabeled samples bring a significant increase in accuracy.

5. Discussions

5.1. Limitations

We discuss the following points for further work and possible room for improvement.

- **Large Number of Unlabeled Data.** GEESOLVER does not rely on a large number of unlabeled samples for recognizing simple captcha schemes. Typically, it requires only 1,000 unlabeled samples for most captcha schemes. However, for captchas with complex security features (e.g., Google and Microsoft), it is important to obtain high-quality latent representation with a large number of unlabeled samples. Since unlabeled captcha schemes can be quickly collected through distributed crawlers, we believe that unlabeled samples are not a key factor hindering our approach.
- **Recognition of Overly Skewed Captchas.** Since the captcha decoder is designed according to the common characteristics of captcha images (i.e., the text is arranged horizontally from left to right), overly skewed captchas are inconsistent with our original intention. The characters in Microsoft captchas of Dataset A are too skewed, almost vertical. When using 1,000 unlabeled samples, GEESOLVER only obtains an accuracy of 22.67%. However, this inconsistent defect can be addressed by more unlabeled samples. With 5,000 unlabeled samples, GEESOLVER achieves an accuracy of 74.76%, far exceeding other state-of-the-arts.

5.2. Countermeasures

To counter our solver, we consider two security mechanisms, i.e., overly skewed text and unpredictable background. As we mentioned in Section 5.1, the overly

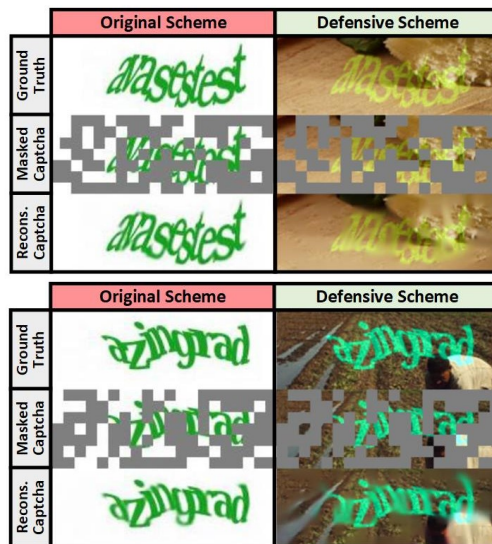


Figure 9. Reconstruction results on Google captcha scheme with unpredictable background. More examples are in the Appendix X.

skewed captcha scheme can limit the performance of solvers trained with a small number of samples (e.g., 1,000 unlabeled samples and 500 labeled samples). However, this countermeasure still fails as more readily available unlabeled samples are introduced for training. Therefore, we design another security feature, unpredictable background, from the perspective of our training paradigm to defend against attacks by our solver.

The unpredictable background is rarely considered in the existing captcha design. The core of GEESOLVER is MAE, which trains the ViT encoder through the reconstruction task of inferring entire characters from their parts. After introducing unpredictable backgrounds, MAE will pay much attention to recovering meaningless and complex backgrounds, which has a huge negative impact on the training of the ViT encoder. To further investigate the reliability of the unpredictable background scheme against GEESOLVER, we compare it with five other common security features. First, we design a captcha generator to synthesize captchas with various security features. Then, we train GEESOLVER with synthetic captchas with the same settings as the experiments to evaluate how security features affect the effectiveness of GEESOLVER. Results are shown in Appendix IX, we can observe that using only the unpredictable background reduces the recognition accuracy even more significantly than using all other security features at the same time, indicating that the unpredictable background is more effective against GEESOLVER. Thus, we combine complex Google captchas with unpredictable backgrounds, and the accuracy drops from 90.7% to 3%. As shown in Figure 9, in the case of the same mask, MAE cannot fully recover the captcha characters due to the interference of complex background noise, which indicates that adding unpredictable backgrounds can prevent MAE from extracting high-quality representations. It can be concluded that the combination of complex captcha

scheme (e.g. Google) with unpredictable background can significantly improve the security of text-based captchas.

Furthermore, we believe that the unpredictable background is also suitable for reducing the effectiveness of other methods for breaking text-based captchas. For segmentation-based attacks, the unpredictable background makes the boundaries between characters less obvious, which can increase the difficulty of character segmentation. For DL-based attacks, the application of unpredictable background means that more interference information needs to be resisted during feature extraction, which will also bring great challenges to DL-based solvers.

6. Related Work

Text-based captchas have suffered various kinds of attacks since they were proposed. To resist existing attack methods, security experts have introduced many security features into captcha design [42]. Text-based captchas and attack methods are undergoing an iterative development process.

Many segmentation-based methods are proposed in the early stage, and rely on preprocessing algorithms to remove noisy backgrounds and segmentation algorithms to separate characters [43]. Based on shape context matching algorithms to find distances between characters, Mori *et al.* [8] successfully broke two early simple text-based captcha schemes. Gao *et al.* [15] used the Log-Gabor filter, a signal processing algorithm, to extract information of individual characters and then used k-Nearest Neighbor algorithm to recognize individual characters. Tang *et al.* [16] designed various preprocessing algorithms and segmentation algorithms for eleven different captcha schemes and leveraged CNNs to effectively recognize individual characters. These segmentation-based methods are tightly coupled to the captcha schemes and are difficult to generalize. The introduction of a new security feature will make the old segmentation algorithm ineffective. Besides, with the emergence of more sophisticated security features, characters are more and more difficult to separate.

Thus, DL-based solvers have been proposed with better recognition performance and generalization capacity. George *et al.* [19] presented a hierarchical model called Recursive Cortical Network (RCN) and broke four schemes with success rates ranging from 57.1% to 66.6%. Zi *et al.* [21] built a captcha-breaking network composed of a CNN and an attention-based RNN. They successfully broke sixteen captcha schemes and the final success rates ranged from 74.8% to 97.3%. However, training DL-based solvers requires a large number of labeled captchas. Once the captcha style is changed or new security features are added, it will take a lot of time and labor to label captchas again. In recent years, some researchers attempted to mitigate the reliance on labeled samples by leveraging synthetic or unlabeled samples. Ye *et al.* [23] utilized the generative adversarial network (GAN) to generate synthetic captchas, which are used to train the CNN-based solver. Tian *et al.* [24] leveraged unlabeled captchas to let the model

predict characters from top to bottom by column with the contrastive learning paradigm. However, their method can only deal with regular and simple captcha; for example, it is difficult to work when the character is not in the same column. Besides, these two methods rely on time-consuming preprocessing algorithms for removing noisy backgrounds. Ye *et al.* leveraged Pix2Pix, an image-to-image translation framework to remove noise and occluding lines, while Tian *et al.* used Double-DIP, an image decomposition approach for removing noisy backgrounds. Besides, both of them fail to recognize the captchas with complex security features. Deng *et al.* [38] proposed to improve the FixMatch framework by combining various advanced ML techniques for training their captcha solver. However, their method lacks attention to the characteristics of text-based captchas, which limits its ability to break difficult captchas and makes it unable to perform well in the case of extremely few labeled samples.

To address the above problems, we propose GEESOLVER, a generic, efficient, and effortless solver to break text-based captchas. The self-supervised MAE-style training paradigm enables our latent representation extractor to extract more efficient representations from the local information of the character. Besides, by taking full advantage of unlabeled samples with self-supervised learning and semi-supervised learning, GEESOLVER mitigates the reliance on labeled samples. Experimental results show GEESOLVER can successfully attack various captcha schemes as well as has a fast recognition speed.

7. Conclusion

In this paper, we proposed a generic, efficient, and effortless solver for breaking text-based captchas. Our solver achieves a breakthrough in breaking captchas with complex security features (e.g., Google). By applying MAE-style self-supervised paradigm in captcha recognition for the first time, we build a latent representation extractor that can extract latent representation from local information, which is of high quality because the whole character can be inferred from it. Extracting the representation from a part of a character that can represent the whole character is the key to breaking difficult captcha schemes, since they employ sophisticated security features to destroy standard characters. To fully exploit the information in unlabeled data, we then train the captcha decoder with a semi-supervised method. Unlike prior deep learning algorithms, our solver requires significantly fewer labeled captchas. Besides, our solver does not rely on any complex preprocessing method and can perform real-time attacks. Most importantly, our solver can recognize various captcha schemes with distinct security features, so its usability will not be affected by newly introduced security features.

Our approach is evaluated on the real-world captcha schemes. Experimental results show that GEESOLVER outperforms four state-of-the-arts with a few labeled captchas. We hope that our work will help security experts to revisit the design and availability of text-based captchas.

Acknowledgments

We are grateful to our shepherd and anonymous reviewers for their constructive comments on this work. Besides, we would like to thank Prof. Haichang Gao and Prof. Zhanyong Tang for providing their datasets for fair comparison. This work is supported by SJTU-QI'ANXIN Joint Lab of Information System Security.

References

- [1] L. v. Ahn, M. Blum, N. J. Hopper, and J. Langford, "Captcha: Using hard ai problems for security," in *International conference on the theory and applications of cryptographic techniques*, 2003, pp. 294–311.
- [2] R. Gossweiler, M. Kamvar, and S. Baluja, "What's up captcha? a captcha based on image orientation," in *Proceedings of the 18th international conference on World wide web*, 2009, pp. 841–850.
- [3] J. Elson, J. R. Douceur, J. Howell, and J. Saul, "Asirra: a captcha that exploits interest-aligned manual image categorization," *CCS*, vol. 7, pp. 366–374, 2007.
- [4] E. Bursztein, R. Beauxis, H. Paskov, D. Perito, C. Fabry, and J. Mitchell, "The failure of noise-based non-continuous audio captchas," in *2011 IEEE symposium on security and privacy*, 2011, pp. 19–31.
- [5] H. Gao, H. Liu, D. Yao, X. Liu, and U. Aickelin, "An audio captcha to distinguish humans from computers," in *2010 Third International Symposium on Electronic Commerce and Security*, 2010, pp. 265–269.
- [6] K. A. Kluever and R. Zanibbi, "Balancing usability and security in a video captcha," in *Proceedings of the 5th Symposium on Usable Privacy and Security*, 2009, pp. 1–11.
- [7] H. Gao, D. Yao, H. Liu, X. Liu, and L. Wang, "A novel image based captcha using jigsaw puzzle," in *2010 13th IEEE International Conference on Computational Science and Engineering*, 2010, pp. 351–356.
- [8] G. Mori and J. Malik, "Recognizing objects in adversarial clutter: Breaking a visual captcha," in *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2003. *Proceedings.*, vol. 1, 2003, pp. I–I.
- [9] J. Yan and A. S. El Ahmad, "Breaking visual captchas with naive pattern recognition algorithms," in *Twenty-Third annual computer security applications conference (ACSAC 2007)*, 2007, pp. 279–291.
- [10] —, "A low-cost attack on a microsoft captcha," in *Proceedings of the 15th ACM conference on Computer and communications security*, 2008, pp. 543–554.
- [11] E. Bursztein, M. Martin, and J. Mitchell, "Text-based captcha strengths and weaknesses," in *Proceedings of the 18th ACM conference on Computer and communications security*, 2011, pp. 125–138.
- [12] H. Gao, W. Wang, J. Qi, X. Wang, X. Liu, and J. Yan, "The robustness of hollow captchas," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 1075–1086.
- [13] H. Gao, X. Wang, F. Cao, Z. Zhang, L. Lei, J. Qi, and X. Liu, "Robustness of text-based completely automated public turing test to tell computers and humans apart," *IET Information Security*, vol. 10, no. 1, pp. 45–52, 2015.
- [14] H. Gao, M. Tang, Y. Liu, P. Zhang, and X. Liu, "Research on the security of microsoft's two-layer captcha," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 7, pp. 1671–1685, 2017.
- [15] H. Gao, J. Yan, C. Fang, Z. Zhang, and J. Li, "A simple generic attack on text captchas," in *Network & Distributed System Security Symposium*, 2016, pp. 1–14.
- [16] M. Tang, H. Gao, Y. Zhang, Y. Liu, P. Zhang, and P. Wang, "Research on deep learning techniques in breaking text-based captchas and designing image-based captcha," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 10, pp. 2522–2537, 2018.
- [17] F. Stark, C. Hazırbas, R. Triebel, and D. Cremers, "Captcha recognition with active deep learning," in *Workshop new challenges in neural computation*, vol. 2015, 2015, p. 94.
- [18] H. Zhan, S. Lyu, and Y. Lu, "Handwritten digit string recognition using convolutional neural network," in *2018 24th International Conference on Pattern Recognition (ICPR)*, 2018, pp. 3729–3734.
- [19] D. George, W. Lehrach, K. Kinsky, M. Lázaro-Gredilla, C. Laan, B. Marthi, X. Lou, Z. Meng, Y. Liu, H. Wang *et al.*, "A generative vision model that trains with high data efficiency and breaks text-based captchas," *Science*, vol. 358, no. 6368, p. eaag2612, 2017.
- [20] T. A. Le, A. G. Baydin, R. Zinkov, and F. Wood, "Using synthetic data to train neural networks is model-based reasoning," in *2017 International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 3514–3521.
- [21] Y. Zi, H. Gao, Z. Cheng, and Y. Liu, "An end-to-end attack on text captchas," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 753–766, 2019.
- [22] C. Li, X. Chen, H. Wang, P. Wang, Y. Zhang, and W. Wang, "End-to-end attack on text-based captchas based on cycle-consistent generative adversarial network," *Neurocomputing*, vol. 433, pp. 223–236, 2021.
- [23] G. Ye, Z. Tang, D. Fang, Z. Zhu, Y. Feng, P. Xu, X. Chen, and Z. Wang, "Yet another text captcha solver: A generative adversarial network based approach," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 332–348.
- [24] S. Tian and T. Xiong, "A generic solver combining unsupervised learning and representation learning for breaking text-based captchas," in *Proceedings of The Web Conference 2020*, 2020, pp. 860–871.
- [25] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," *arXiv preprint arXiv:2111.06377*, 2021.
- [26] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations (ICLR)*, 2021.
- [27] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," *Advances in Neural Information Processing Systems*, vol. 33, pp. 596–608, 2020.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [30] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [31] P. Bachman, R. D. Hjelm, and W. Buchwalter, "Learning representations by maximizing mutual information across views," *Advances in neural information processing systems*, vol. 32, 2019.
- [32] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. E. Hinton, "Big self-supervised models are strong semi-supervised learners," *Advances in neural information processing systems*, vol. 33, pp. 22 243–22 255, 2020.

- [33] X. Chen, H. Fan, R. Girshick, and K. He, “Improved baselines with momentum contrastive learning,” *arXiv preprint arXiv:2003.04297*, 2020.
- [34] J. Baek, G. Kim, J. Lee, S. Park, D. Han, S. Yun, S. J. Oh, and H. Lee, “What is wrong with scene text recognition model comparisons? dataset and model analysis,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 4715–4723.
- [35] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, “On the properties of neural machine translation: Encoder-decoder approaches,” in *Proceedings of SSST@EMNLP 2014, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, D. Wu, M. Carpuat, X. Carreras, and E. M. Vecchi, Eds., 2014, pp. 103–111.
- [36] L. Likforman-Sulem, A. Zahour, and B. Taconet, “Text line segmentation of historical documents: a survey,” *International Journal of Document Analysis and Recognition (IJ DAR)*, vol. 9, no. 2, pp. 123–138, 2007.
- [37] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE transactions on systems, man, and cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [38] X. Deng, R. Zhao, Y. Wang, L. Chen, Y. Wang, and Z. Xue, “3E-Solver: An effortless, easy-to-update, and end-to-end solver with semi-supervised learning for breaking text-based captchas,” in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 2022, pp. 3817–3824.
- [39] T. Wang, Y. Zhu, L. Jin, C. Luo, X. Chen, Y. Wu, Q. Wang, and M. Cai, “Decoupled attention network for text recognition,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, 2020, pp. 12 216–12 224.
- [40] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International conference on machine learning*, 2020, pp. 1597–1607.
- [41] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar *et al.*, “Bootstrap your own latent-a new approach to self-supervised learning,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 271–21 284, 2020.
- [42] Y. Zhang, H. Gao, G. Pei, S. Luo, G. Chang, and N. Cheng, “A survey of research on captcha designing and breaking techniques,” in *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, 2019, pp. 75–84.
- [43] K. Chellapilla, K. Larson, P. Y. Simard, and M. Czerwinski, “Computers beat humans at single character recognition in reading based human interaction proofs (hips).” in *CEAS*, 2005.
- [44] I. Loshchilov and F. Hutter, “Fixing weight decay regularization in adam,” 2018.

Appendix

I. Algorithm

Algorithm 1 The Pipeline of Two-Stage Training Framework

Input: Labeled dataset, unlabeled dataset, latent representation extractor f_1 , reconstruction decoder g , captcha decoder f_2 , the number of iterations in Stage I T_1 and Stage II T_2 .

Output: Trained models f_1 and f_2 .

- 1: **for** $i = 1$ to T_1 **do**
 - 2: Sample an unlabeled image x
 - 3: Employ a data augmentation on x
 - 4: Randomly mask patches and denote visible patches as x_{vis}
 - 5: Extract representation of visible patches: $z_{vis} = f_1(x_{vis})$
 - 6: Reconstruct Missing patches: $y_{rec} = g(z_{vis})$
 - 7: Compute reconstruction loss: $\mathcal{L}_{rec} = MSE(y_{rec}, y_{real})$
 - 8: Update f_1 using gradient descent algorithm
 - 9: **end for**
 - 10: Frozen parameters in f_1
 - 11: **for** $i = 1$ to T_2 **do**
 - 12: Sample a labeled image x and an unlabeled image x_u
 - 13: Supervised loss: $\mathcal{L}^s = H(f_2(f_1(x)), y)$
 - 14: Pseudo-label: $\hat{q} = \operatorname{argmax}(f_2(f_1(x_u^W)))$
 - 15: Unsupervised loss: $\mathcal{L}^u = \mathbb{1}(\max(q) \geq \tau) H(f_2(f_1(x_u^S)), \hat{q})$
 - 16: Total loss: $\mathcal{L}_{total} = \mathcal{L}^s + \mathcal{L}^u$
 - 17: Update f_2 using gradient descent algorithm
 - 18: **end for**
 - 19: **return** f_1 and f_2
-

II. Captcha Augmentation Methods

To improve the quality of learned latent representation, we adopt augmentation methods for captcha image to improve the difficulty of reconstruction, including (1) *AutoContrast*, (2) *Brightness*, (3) *Color*, (4) *Contrast*, (5) *Equalize*, (6) *Posterize*, (7) *Rotate*, (8) *Sharpness*, (9) *Shear*, (10) *Solarize*, (11) *Translate*, (12) *Distort*, (13) *Stretch*, and (14) *Perspective*, as shown in Figure A1.

III. Our Captcha Dataset

Text-based captcha remains the most extensively used security solution by many websites (e.g., Google, Yandex, and Microsoft) and the web services of IoT devices (e.g. ASUS router). For example, Google employs text-based captchas on the site <https://accounts.google.com/>, after multiple consecutive incorrect passwords are entered, the



Figure A1. Fourteen data augmentation methods adopted in this paper for improving the difficulty of reconstruction.

user will be required to submit both the password and the results of text-based captchas. Our dataset contains eight text-based captcha schemes of the top-50 popular websites ranked by *Alexa*, including Google, Yandex, Microsoft, Apple, Sina, Weibo, Wikipedia, and Ganji. We carefully made our datasets according to the following strict standards.

- 1) We collected 7,000 captcha samples from each website.
- 2) Then, we deleted duplicate images using MD5 Message-Digest Algorithm.
- 3) Next, 2,000 captcha images are labeled for each scheme,
- 4) Finally, we randomly selected 500 labeled captcha images as training set and the other 1500 as the test set. Besides, the unlabeled subset consists of 5,000 unlabeled captcha images.

Except for the experiments of comparison with prior works, other experiments are all performed on our dataset.

IV. Implementation Details

In Stage I, we train the latent representation extractor with an AdamW optimizer [44], and the learning rate is set to $1.5 \times e^{-4}$. The ViT encoder has 8 Transformer blocks, and the reconstruction decoder has 4 Transformer blocks. The masking ratio for randomly masking patches is set to 0.6. In Stage II, we train the captcha decoder with an SGD optimizer, and the learning rate is set to $2 \times e^{-2}$. The confidence threshold of the FixMatch is set to 0.95. We employ continuous information compression module by default. The MAE is trained for 600k iterations, and the captcha decoder is trained for 100k iterations. The proposed approach is implemented using PyTorch 1.9.0 and trained on the PC with Intel® Core™ i9-11900K@3.50 GHz, 64 GB RAM, and an NVIDIA GeForce RTX3090 GPU.

TABLE A1. THE RESULTS OF TRAINING GEESOLVER WITH ALL COLLECTED CAPTCHA SAMPLES FOR GENERALITY EVALUATION.

Scheme	Specific	Mixed	Scheme	Specific	Mixed
Google	90.7%	72.7%	Weibo	92.5%	92.5%
Yandex	92.9%	82.9%	Sina	97.0%	96.7%
Microsoft	97.4%	88.2%	Apple	95.6%	92.0%
Wikipedia	97.8%	96.7%	Ganji	99.7%	98.9%

V. Generality Analysis

The characters in different captcha schemes have different fonts, colors, and distortions, so different captcha schemes may interfere with each other. The results of training GEESOLVER with all collected captcha samples for generality evaluation are shown in Table A1. We can observe that accuracy of training on all collected captcha samples is lower than that of the specific captcha scheme, but still has good performance. Also, we note that the original MAE is trained on 10 million images. Therefore, introducing a larger-scale unlabeled captcha samples may further improve the performance.

VI. Evaluation Protocol for Comparison

Thanks to two datasets provided by Tang *et al.* [16] and Ye *et al.* [23], we can make fair comparisons with their methods. Since Zi *et al.* [21] reported their results on the dataset A and Tian *et al.* [24] reported their results on both datasets, we compare our approach with these four state-of-the-arts on these two datasets. Tang *et al.* [16] provided dataset A and used 2000 labeled samples for training. Ye *et al.* [23] provided dataset B and used 500 labeled samples for training. Tian *et al.* [24] used the same number of labeled samples with them (i.e. 2000 for dataset A, 500 for dataset B) and used additional 1000 unlabeled samples for training. Zi *et al.* [21] use different numbers of labeled samples (i.e., 2000, 4000, 6000, 8000, and 10000) in dataset A for training an end-to-end model. To be more in line with the motivation of our work, we leverage only 500 labeled samples for both datasets and use additional 1000 or 5000 unlabeled samples to train our solver.

VII. Reconstruction Results on Eight Captcha Schemes

To achieve a qualitative sense of the reconstruction task in Stage I, we randomly mask 3 captcha images from the test set of each captcha scheme, and then reconstruct them. Examples of masked captcha, reconstructed captcha, and ground truth captcha are shown in TABLE A2. It can be seen that even if the captcha is masked with a high masking ratio, the reconstruction task can still be completed well. Furthermore, we observe some reconstructed images that do not match the ground truth but are still plausible. This reasoning-like behavior indicates that our ViT encoder can extract high-quality latent representation.

TABLE A2. RECONSTRUCTION RESULTS ON *test* SET OF EIGHT CAPTCHA SCHEMES. For each triplet, we exhibit the masked captcha (left), reconstructed captcha (middle), and the ground-truth (right). The masking ratio is 60%, leaving only 56 out of 140 patches. The training schedule length is 600k iterations.

Scheme	Masked Captcha	Reconstructed Captcha	Ground Truth	Scheme	Masked Captcha	Reconstructed Captcha	Ground Truth
Google				Weibo			
Yandex				Sina			
Microsoft				Apple			
Ganji				Wikipedia			

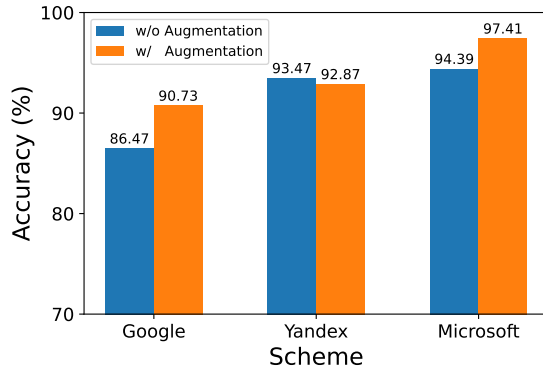


Figure A2. The influence of data augmentation on the quality of learned representation.

VIII. Impact of Captcha Image Augmentation

To improve the quality of learned latent representation, we adopt some augmentation methods for captchas to improve the difficulty of reconstruction (see Appendix II). The experiments results are shown in Figure A2. For Google and Microsoft captchas, data augmentation methods further increase the variety of characters, which helps the latent representation extractor to better learn the invariant

representation of variant characters. For *Yandex* captcha scheme, because of its hollow font and thin font line, there is no obvious difference between augmentation and non-augmentation.

IX. Impacts of Captcha Security Features

The impacts of different captcha security features are shown in TABLE A3. We design a captcha generator to synthesize captchas with various security features and train GEESOLVER with synthetic captchas with the same settings as the experiments. It can be concluded that the unpredictable background is the most effective security feature against our solver.

X. Reconstruction Results on on Google Captchas with Unpredictable Background

The reconstruction results of the Google captchas with and without unpredictable backgrounds are shown in Figure A3. For each triplet, we exhibit the ground truth (top), the masked captcha (middle), and the reconstructed captcha (bottom). It can be seen that the unpredictable background significantly reduces the model's ability to reconstruct characters, indicating the effectiveness of this novel security feature.

TABLE A3. IMPACTS OF SECURITY FEATURES.

No.	Example	Curve	Rotation	Perspective Transform	Distortion	Noisy	Unpredictable Background	CCT	Accuracy
0	aaarigaxb	✗	✗	✗	✗	✗	✗	✗	97.87%
1	aacujnbc	✓	✗	✗	✗	✗	✗	✗	97.67%
2	aadmekwp	✓	✓	✗	✗	✗	✗	✗	93.93%
3	abneetfaci	✓	✓	✓	✗	✗	✗	✗	92.00%
4	fradlegmx	✓	✓	✓	✓	✗	✗	✗	91.04%
5	netrvleqn	✓	✓	✓	✓	✓	✗	✗	88.20%
6	aasdyztsbi	✗	✗	✗	✗	✗	✓	✗	71.93%
7	arises	✗	✓	✗	✓	✗	✗	✓	90.73%
8	arises	✗	✓	✗	✓	✗	✓	✓	3.00%

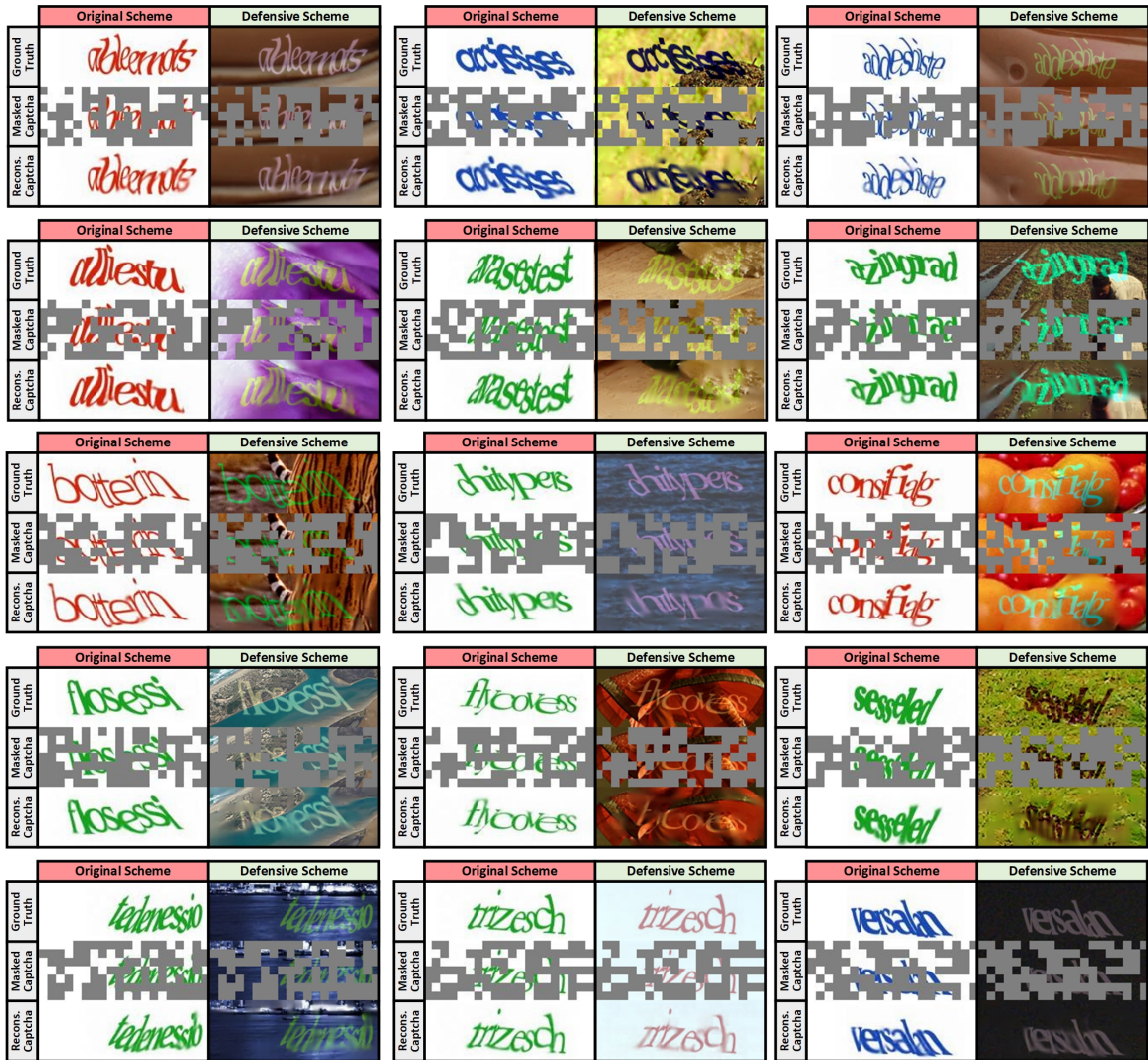


Figure A3. Reconstruction results on Google captcha scheme with unpredictable background.